

1. Objective

To implement BFS and DFS on a graph using an adjacency matrix, with the queue and stack built manually using arrays, and to correctly handle disconnected graphs.

2. Theory and Algorithm

2.1 Breadth-First Search (BFS)

BFS explores a graph level by level. Starting from a given node, it first visits all of that node's direct neighbours before moving on to the next level of neighbours. This behaviour is achieved using a queue, which follows First-In-First-Out (FIFO) order.

BFS Algorithm:

1. Mark the starting node as visited and insert it into the queue.
2. While the queue is not empty, remove (dequeue) the front node and print it.
3. For every neighbour of the removed node, if it is not visited, mark it visited and insert it into the queue.
4. Repeat step 2 and 3 until the queue becomes empty.
5. If any node is still unvisited after this (disconnected graph), repeat the process from that node.

2.2 Depth-First Search (DFS)

DFS explores a graph by going as deep as possible along a branch before backtracking. This is achieved using a stack, which follows Last-In-First-Out (LIFO) order. DFS also records a start time (when a node is first discovered) and a finish time (when a node and all of its descendants have been fully explored).

DFS Algorithm:

1. Mark the starting node as visited, push it onto the stack, and record its start time.
2. Look at the node on top of the stack. Find its next unvisited neighbour.
3. If an unvisited neighbour is found, mark it visited, push it onto the stack, and record its start time.
4. If no unvisited neighbour is left for the top node, pop it from the stack and record its finish time.
5. Repeat step 2 to 4 until the stack becomes empty.
6. If any node is still unvisited after this (disconnected graph), repeat the process from that node.

3. Data Structures Used

3.1 Adjacency Matrix

The graph is represented using a 2D array `adj[MAX][MAX]`, where `adj[i][j] = 1` indicates an edge from node `i` to node `j`, and `adj[i][j] = 0` indicates no edge. This representation makes it simple to check whether an edge exists between any two nodes in constant time.

3.2 Manual Queue (Array-Based, Circular)

The Queue class uses a fixed-size array along with front and rear indices. It supports `enqueue()` and `dequeue()` operations, and wraps around circularly using the modulus operator so array space is reused efficiently instead of shifting elements.

3.3 Manual Stack (Array-Based)

The Stack class uses a fixed-size array along with a top index. It supports `push()`, `pop()`, and `peek()` operations, where `peek()` allows checking the top element of the stack without removing it. This is used in DFS to keep exploring from the current node before backtracking.

4. Handling Disconnected Graphs

A graph is disconnected when it is not possible to reach every node from a single starting node. To handle this, the `visited[]` array is kept global so its values persist across multiple calls to `BFS()` or `DFS()`. After the traversal from the given starting node finishes, the program loops through every node from 1 to `n`. If a node is still marked as unvisited, it means that node belongs to a separate component of the graph that was not reachable from the original starting node. The program then calls `BFS()` or `DFS()` again, this time starting from that unvisited node. This process repeats until every node in the graph has been visited, ensuring that all components of a disconnected graph are fully traversed.

The program also prints a short message such as "(node X was in a disconnected part, starting new BFS/DFS)" whenever this happens, so the disconnected handling is visible directly in the program's output.

5. Source Code

File name: 270_BFS_DFS.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  const int MAX = 100; // max nodes allowed
5
6  int visited[MAX]; // global visited array, shared across BFS/DFS calls
7  int startTime[MAX]; // start time for each node (DFS)
8  int finishTime[MAX]; // finish time for each node (DFS)
9  int time_counter = 0; // timer used in DFS
10
11 // Queue class for BFS
12 class Queue
13 {
14     int arr[MAX]; // holds queue elements
15     int front, rear; // front and rear index
16
17 public:
18     Queue()
19     {
20         front = rear = -1; // empty queue at start
21     }
22
23     bool isEmpty()
24     {
25         return front == -1; // queue empty if front is -1
26     }
27
28     bool isFull()
29     {
30         return (rear + 1) % MAX == front; // circular full check
31     }
32
33     void enqueue(int val)
34     {
35         if (isFull()) // can't add if full
36         {
37             cout << "Queue is full\n";
38             return;
39         }
40
41         if (isEmpty()) // first element
42             front = rear = 0;
43         else
44             rear = (rear + 1) % MAX; // move rear forward
45
46         arr[rear] = val; // insert value
47     }
48
49     int dequeue()
50     {
51         if (isEmpty()) // nothing to remove
52             return -1;
53
54         int val = arr[front]; // value to return
55
56         if (front == rear) // only one element was left
57             front = rear = -1;
58         else
59             front = (front + 1) % MAX; // move front forward
```

```

60
61     return val;
62 }
63 };
64
65 // Stack class for DFS
66 class Stack
67 {
68     int arr[MAX]; // holds stack elements
69     int top; // index of top element
70
71 public:
72     Stack()
73     {
74         top = -1; // empty stack at start
75     }
76
77     bool isEmpty()
78     {
79         return top == -1; // stack empty if top is -1
80     }
81
82     bool isFull()
83     {
84         return top == MAX - 1; // full when top hits last index
85     }
86
87     void push(int val)
88     {
89         if (isFull()) // can't add if full
90         {
91             cout << "Stack is full\n";
92             return;
93         }
94
95         arr[++top] = val; // move top up, then insert
96     }
97
98     int pop()
99     {
100         if (isEmpty()) // nothing to remove
101             return -1;
102
103         return arr[top--]; // return top value, then move top down
104     }
105
106     int peek()
107     {
108         if (isEmpty()) // nothing on top
109             return -1;
110
111         return arr[top]; // just look at top value
112     }
113 };
114
115 // BFS from a single starting node (SN), same steps as taught in class
116 void BFS(int adj[MAX][MAX], int n, int SN)
117 {
118     Queue q; // queue for BFS
119
120     q.enqueue(SN); // put start node in queue
121     visited[SN] = 1; // mark it visited
122
123     while (!q.isEmpty()) // keep going till queue is empty

```

```

124     {
125         int cur = q.dequeue(); // take front node out
126
127         cout << cur + 1 << " "; // print node (1-based)
128
129         for (int i = 0; i < n; i++) // check every possible neighbour
130         {
131             if (adj[cur][i] == 1 && visited[i] != 1) // edge exists and not visited
132             {
133                 q.enqueue(i); // add neighbour to queue
134                 visited[i] = 1; // mark visited
135             }
136         }
137     }
138 }
139
140 // DFS from a single starting node (SN), iterative using stack
141 // also records start time and finish time for each node
142 void DFS(int adj[MAX][MAX], int n, int SN)
143 {
144     int nextChild[MAX] = {0}; // next neighbour to check for each node
145
146     Stack st; // stack for DFS
147
148     st.push(SN); // push starting node
149     visited[SN] = 1; // mark visited
150     startTime[SN] = ++time_counter; // record start time
151     cout << SN + 1 << " "; // print node (1-based)
152     nextChild[SN] = 0; // begin checking from neighbour 0
153
154     while (!st.isEmpty())
155     {
156         int cur = st.peek(); // look at node on top without removing
157         bool found = false; // did we find a new node to go into
158
159         while (nextChild[cur] < n) // check remaining neighbours of cur
160         {
161             int next = nextChild[cur]; // neighbour to check
162             nextChild[cur]++; // move pointer so we don't recheck this one
163
164             if (adj[cur][next] == 1 && visited[next] != 1) // valid unvisited neighbour
165             {
166                 st.push(next); // go deeper into this neighbour
167                 visited[next] = 1; // mark visited
168                 startTime[next] = ++time_counter; // record start time
169                 cout << next + 1 << " "; // print node (1-based)
170                 nextChild[next] = 0; // reset neighbour pointer for new node
171                 found = true; // we moved to a new node
172                 break; // stop checking, go back to while loop with new top
173             }
174         }
175
176         if (!found) // no more unvisited neighbours left for cur
177         {
178             st.pop(); // remove it from stack, it's fully done
179             finishTime[cur] = ++time_counter; // record finish time
180         }
181     }
182 }
183
184 // takes edges from user and fills adjacency matrix
185 void inputEdgesToAdjMatrix(int adj[MAX][MAX], int n, int e)
186 {
187     // Initialize matrix with 0

```

```

188     for (int i = 0; i < n; i++)
189         for (int j = 0; j < n; j++)
190             adj[i][j] = 0;
191
192     cout << "Enter edges (source destination):\n";
193
194     for (int i = 0; i < e; i++)
195     {
196         int u, v; // stores one edge (source, destination)
197         cin >> u >> v; // read one edge
198
199         // check nodes are in range, not same node, and edge not already there
200         if (u >= 1 && u <= n &&
201             v >= 1 && v <= n &&
202             u != v &&
203             adj[u - 1][v - 1] == 0)
204         {
205             adj[u - 1][v - 1] = 1; // mark edge in matrix
206         }
207         else
208         {
209             cout << "Invalid edge\n";
210             i--; // ask for same edge number again
211         }
212     }
213 }
214
215 // prints the adjacency matrix
216 void printAdjMatrix(int adj[MAX][MAX], int n)
217 {
218     cout << "\nAdjacency Matrix:\n";
219
220     for (int i = 0; i < n; i++)
221     {
222         for (int j = 0; j < n; j++)
223             cout << adj[i][j] << " "; // print each cell
224
225         cout << endl; // new row
226     }
227 }
228
229 int main()
230 {
231     int n, e; // number of nodes and number of edges
232     int adj[MAX][MAX]; // adjacency matrix for the graph
233
234     cout << "Enter number of nodes: ";
235     cin >> n;
236
237     while (n <= 0 || n > MAX) // keep asking until valid
238     {
239         cout << "Invalid input, enter again: ";
240         cin >> n;
241     }
242
243     cout << "Enter number of edges: ";
244     cin >> e;
245
246     while (e < 0 || e > n * (n - 1)) // max possible edges without self loops
247     {
248         cout << "Invalid input, enter again: ";
249         cin >> e;
250     }
251

```

```

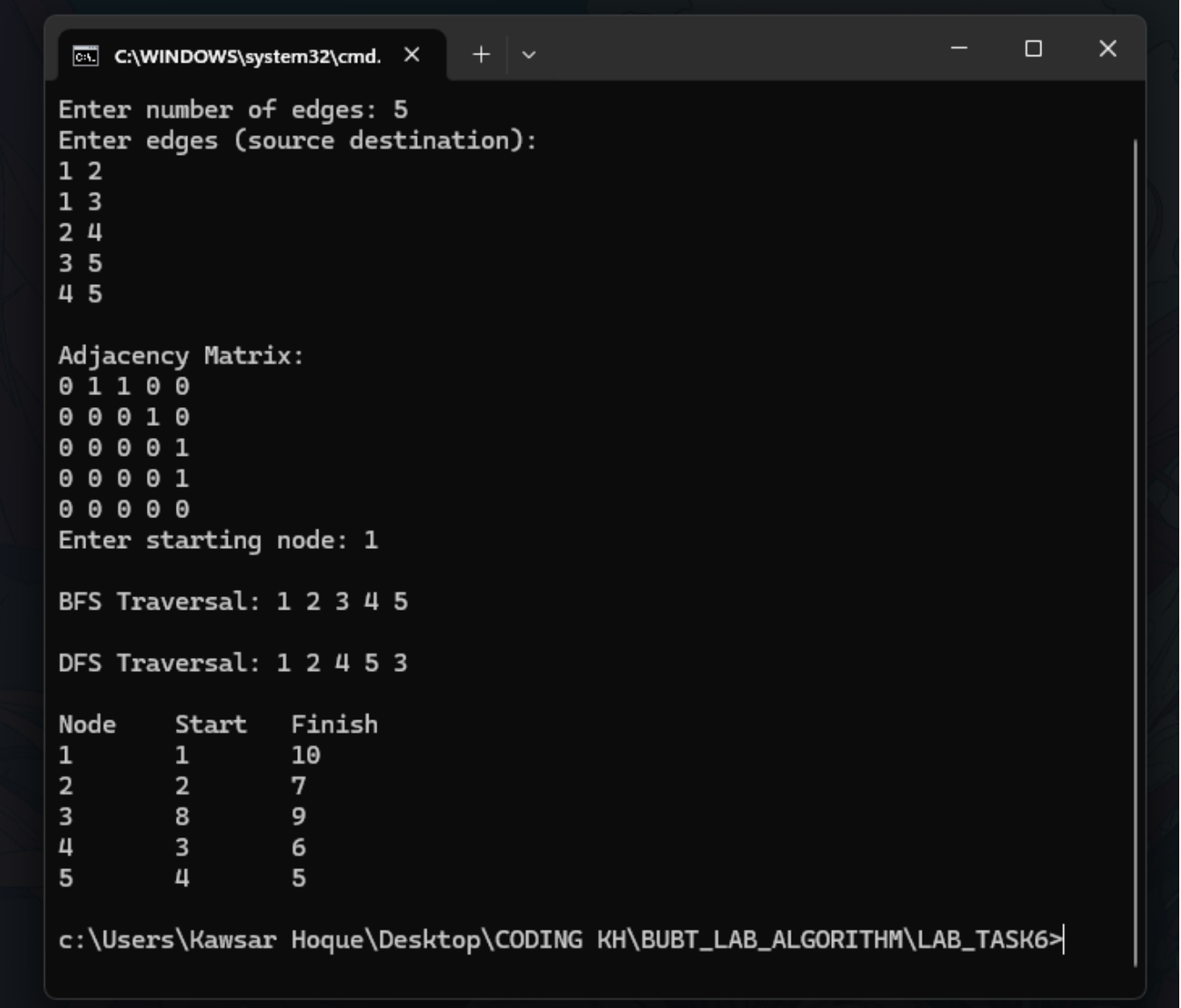
252     inputEdgesToAdjMatrix(adj, n, e); // build the matrix
253
254     printAdjMatrix(adj, n); // show matrix to user
255
256     int start; // node the user wants to start traversal from
257
258     cout << "Enter starting node: ";
259     cin >> start;
260
261     while (start < 1 || start > n) // keep asking until valid
262     {
263         cout << "Invalid input, enter again: ";
264         cin >> start;
265     }
266
267     // ---- BFS ----
268     for (int i = 0; i < n; i++) // reset visited before BFS
269         visited[i] = 0;
270
271     cout << "\nBFS Traversal: ";
272     BFS(adj, n, start - 1); // BFS from the given start node
273
274     // this loop covers disconnected nodes, same as taught in class:
275     // if any node is still not visited after the first BFS call,
276     // it means it belongs to a separate component, so we call
277     // BFS again starting from that node
278     for (int i = 0; i < n; i++)
279     {
280         if (visited[i] != 1)
281         {
282             cout << "\n(node " << i + 1 << " was in a disconnected part, starting new BFS) ";
283             BFS(adj, n, i);
284         }
285     }
286
287     cout << endl;
288
289     // ---- DFS ----
290     for (int i = 0; i < n; i++) // reset visited before DFS
291         visited[i] = 0;
292     time_counter = 0; // reset timer
293
294     cout << "\nDFS Traversal: ";
295     DFS(adj, n, start - 1); // DFS from the given start node
296
297     // same disconnected handling for DFS
298     for (int i = 0; i < n; i++)
299     {
300         if (visited[i] != 1)
301         {
302             cout << "\n(node " << i + 1 << " was in a disconnected part, starting new DFS) ";
303             DFS(adj, n, i);
304         }
305     }
306
307     cout << endl;
308
309     // print start and finish time table for DFS
310     cout << "\nNode\tStart\tFinish\n";
311     for (int i = 0; i < n; i++)
312         cout << i + 1 << "\t" << startTime[i] << "\t" << finishTime[i] << endl;
313
314     return 0;
315 }

```

```
316
317 /*
318 5
319 5
320 1 2
321 1 3
322 2 4
323 3 5
324 4 5
325 1
326 */
```

6. Sample Input and Output

The screenshots below show the program compiled and executed for both a connected graph and a disconnected graph, including the adjacency matrix, BFS traversal, DFS traversal, and the DFS start/finish time table.



```
C:\WINDOWS\system32\cmd. x + v
Enter number of edges: 5
Enter edges (source destination):
1 2
1 3
2 4
3 5
4 5

Adjacency Matrix:
0 1 1 0 0
0 0 0 1 0
0 0 0 0 1
0 0 0 0 1
0 0 0 0 0
Enter starting node: 1

BFS Traversal: 1 2 3 4 5

DFS Traversal: 1 2 4 5 3

Node      Start  Finish
1         1      10
2         2       7
3         8       9
4         3       6
5         4       5

c:\Users\Kawsar Hoque\Desktop\CODING KH\BUBT_LAB_ALGORITHM\LAB_TASK6>
```